🚀 Data Engineering Daily – Day 27

⚡ *The WHERE NOT IN Disaster — Why It Slows Down Spark Jobs*

Page 1: The Challenge — A Simple Query That Breaks Big Data 💣

🎯 **Scenario:**
You're cleaning up unprocessed records in a large data pipeline.
You write a simple SQL query:

```
SELECT *
FROM orders
WHERE order_id NOT IN (SELECT order_id FROM processed_orders);
```

Looks harmless, right?
But when you run this on Spark with:
📁 `orders` = 300 GB
📁 `processed_orders` = 50 GB

Your job starts **crawling** 🐌 or **blows up** with OOM errors.

💥 **Why this happens:**
`NOT IN` isn't a simple filter — it's a **full comparison** between both datasets.
Spark must check *every record* from `orders` against *every value* in `processed_orders`.
Even worse, it must handle `NULL` values carefully because:

> If any `NULL` exists in the subquery, the whole comparison turns unknown.

So Spark can't short-circuit or optimize this — it has to **shuffle and scan everything**.

⚠️ Result:

- High shuffle read/write I/O
- Massive memory use on driver
- Slow join stage performance
- Occasional driver OOM

DAILY DATA ENGINEERING BY MAYANK

Page 2: The Root Cause — How Spark Handles `NOT IN` Internally 🧠

Let's look at what happens inside Spark's physical plan 👇

1️⃣ **Spark rewrites `NOT IN` as a filter + cross-check:**

```
Filter NOT (order_id IN (SELECT ...))
```

2️⃣ It tries to collect the subquery (`processed_orders`) to the driver.

- If small → collected as an array (broadcast).
- If large → triggers a **shuffle join** (ouch).

3️⃣ It must evaluate each `order_id` row one by one, since `NULL`s break deterministic filtering.

💡 **Key Insight:**

> `NOT IN` doesn't scale well because Spark can't apply broadcast or partition pruning safely.

✅ If your dataset fits in memory → fine.
❌ If it's multi-GB → disaster.

🧩 **Real Example:**

Dataset	Size	Duration (NOT IN)	Duration (LEFT ANTI)
orders (300GB) + processed (50GB)		**1h 42m**	**23m**

That's a **4× performance boost** just by changing the logic! ⚡

DAILY DATA ENGINEERING BY MAYANK

Page 3: The Fix — Anti Joins to the Rescue 🧩

🧠 Instead of checking each record individually, let Spark *use its join engine*.

✅ Better Approach 1 – LEFT ANTI JOIN

```
df_final = orders.join(processed_orders, "order_id", "left_anti")
```

- 🔹 Keeps only records that don't match.
- 🔹 Avoids `NULL` confusion.
- 🔹 Efficiently uses shuffles + partition pruning.

💡 Why it's faster:

- Spark distributes join keys evenly.
- Uses optimized join execution.
- Doesn't have to load the subquery to the driver.

✅ Better Approach 2 – NOT EXISTS (SQL)

```
SELECT * FROM orders o
WHERE NOT EXISTS (
  SELECT 1 FROM processed_orders p
  WHERE o.order_id = p.order_id
);
```

💬 Equivalent logic to LEFT ANTI JOIN,
but cleaner for SQL-based pipelines.

📊 In Databricks:

`NOT EXISTS` → optimized into a **left anti join** internally.

DAILY DATA ENGINEERING BY MAYANK

Page 4: Deep Dive — NOT IN vs. NOT EXISTS vs. LEFT ANTI JOIN 🧠

Type	Works with NULLs?	Optimized by Spark?	Best For	Notes
NOT IN	❌ Fails with NULLs	❌ No	Small tables only	Triggers full scan
NOT EXISTS	✅ Yes	✅ Yes	SQL-based pipelines	Logical anti-join
LEFT ANTI JOIN	✅ Yes	✅ Yes	PySpark/DataFrame APIs	Fastest & most scalable

---

💡 **Best Practice Tips:**
1️⃣ Use **LEFT ANTI JOIN** for large dataframes (> few GB).
2️⃣ Always ensure `join` key columns are of same type.
3️⃣ Add partition filters before joining (reduces shuffle size).
4️⃣ Cache the smaller DataFrame if reused multiple times.
5️⃣ Monitor Spark UI → *Join Stage Size* + *Shuffle Write* after optimization.

📘 **Example Performance Tuning:**

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 50 * 1024 * 1024)

orders.join(processed_orders, "order_id", "left_anti") \
    .repartition(200) \
    .write.format("delta").mode("overwrite").save("/mnt/clean_orders")
```

✅ Reduced shuffle
✅ Controlled partition size
✅ Faster write stage

DAILY DATA ENGINEERING BY MAYANK

Page 5: Interview-Ready Summary 🎯

💬 **How to Explain in an Interview:**

> "In Spark, `NOT IN` can trigger expensive cross-checks and shuffle operations, especially when subqueries contain NULLs or large datasets.
> I prefer using `LEFT ANTI JOIN` or `NOT EXISTS`, which translate into efficient anti-join operations and scale well for large datasets."

🎓 **Key Takeaways:**

- ❌ `NOT IN` = Expensive, unsafe with NULLs
- ✅ `LEFT ANTI JOIN` = Scalable, parallel-friendly
- ✅ `NOT EXISTS` = Safe, equivalent for SQL pipelines
- ⚙️ Tune `autoBroadcastJoinThreshold` for smaller reference tables
- 📊 Always check Spark UI for shuffle size

DAILY DATA ENGINEERING BY MAYANK

✨ **Part of my 100 Days of Data Engineering Series**
👉 Stay tuned for **Day 28 → Day 100**,
where we'll keep uncovering real performance stories from Spark & Databricks.